Roadmap du conseiller IA local

Oct 21, 2026 · @Ethan

Un assistant 100 % local (Ollama sur ta RTX 4070) piloté par FastAPI et LangGraph, avec une interface React, construit à deux en 8 phases d'environ 2 à 4 semaines chacune, à 6 h par semaine de ton côté.

Le nom : à changer avant le premier commit

Le nom de travail, lu par un anglophone ou prononcé à voix haute, sonne comme une insulte raciste. Sur un CV, c'est la première ligne du projet : un recruteur la lit avant l'architecture, et vous n'aurez pas l'occasion d'expliquer l'acronyme. Je n'ai pas de chiffre sur ce risque, mais il ne dépend pas de votre intention, et un dépôt public est vu par tous ceux qui ouvrent le lien.

Aujourd'hui, le changer ne coûte rien. Dans quelques semaines, il faudrait le renommer partout : dépôt GitHub, paquet Python (`import ...`), variables d'environnement, README, captures d'écran, historique Git.

Proposition : choisissez à deux un nom court, prononçable en français et en anglais, et libre sur GitHub. En attendant, ce document appelle le projet « le conseiller », et `conseiller` dans le code.

La machine d'Ethan : ce que 12 Go de VRAM permettent

Le modèle tourne sur le PC d'Ethan : RTX 4070 avec 12 Go de GDDR6X, et 32 Go de DDR5. Deux choses se partagent la VRAM :

- **les poids du modèle**, fixes une fois le modèle chargé ;
- **le cache KV**, qui grandit avec la longueur du contexte (détail dans la vigilance VRAM plus bas).

Windows et l'écran en occupent déjà une part : `nvidia-smi` dans un terminal montre combien.

Les modèles d'Ollama sont **quantifiés** : chaque poids est stocké sur environ 4 bits au lieu de 16, ce qui divise la taille par près de 4 pour une perte de qualité faible. `Q4_K_M` est le format le plus courant.

Taille du modèle	Poids en Q4_K_M (ordre de grandeur)	Place restante pour le contexte
7 à 8 milliards de paramètres	~5 Go	Confortable : 16k à 32k tokens
12 à 14 milliards	~8 à 9 Go	Serrée : 8k tokens environ, selon le modèle
30 milliards et plus	plus de 12 Go	Aucune : le modèle déborde sur la RAM et devient très lent

Recommandation : partez sur un modèle de 7 à 8 milliards de paramètres qui sait appeler des outils (sur ollama.com/library, l'étiquette « tools »). Des agents ont besoin de contexte plus que de taille. Gardez un 14B pour comparer plus tard, pas comme base.

Un seul modèle pour tous les agents. Pour Ollama, un « agent » n'est qu'un prompt et une liste d'outils envoyés au même modèle : dix agents ne coûtent pas plus de VRAM qu'un seul. Deux modèles différents, en revanche, veulent dire deux chargements, ou un échange disque-VRAM de plusieurs secondes à chaque changement.

Le modèle d'embeddings du RAG (par exemple `nomic-embed-text`, moins de 300 Mo) est petit et cohabite sans problème.

Les 32 Go de RAM servent au backend, au front, à la base vectorielle, et de débordement. Mais un modèle qui ne tient pas entier en VRAM génère plusieurs fois plus lentement : `ollama ps` doit afficher « 100% GPU ».

DaVinci Resolve utilise lui aussi beaucoup la VRAM. Je n'ai pas de mesure pour cette configuration : c'est à mesurer en phase 8, avec la méthode de la vigilance VRAM.

Travailler à deux : un dépôt, un contrat, deux rôles

Le dépôt

- **Un seul dépôt GitHub privé** (un « monorepo ») qui contient `backend/` et `frontend/` : une seule CI, un seul historique à montrer. Passez-le en public le jour où vous voulez le montrer, après avoir vérifié qu'aucun secret n'y est jamais entré.
- **`main` protégée** : personne n'y pousse directement. Une tâche = une branche `feat/...` = une pull request, relue et approuvée par l'autre. C'est la règle qui vous fera le plus apprendre : chacun lit tout le code de l'autre, et vous pourrez tous les deux expliquer l'ensemble en entretien.

- **Les tâches** vivent dans GitHub Issues, avec un tableau GitHub Projects (À faire, En cours, En revue, Fait). Une issue = une chose vérifiable en une ou deux séances.
- **`.gitignore` et `.env.example` dès le premier commit** : les vraies clés (`.env`, jetons Google) ne sont jamais versionnées.
- **`docs/decisions/`** : un fichier Markdown court par choix important (pourquoi LangGraph, pourquoi Vite, pourquoi Chroma). En entretien, les questions commencent par « pourquoi » : les réponses seront écrites.

Les outils

Côté	Outil	Rôle
Python	uv	Environnement virtuel et dépendances, verrouillées dans `uv.lock` : vous avez exactement les mêmes versions
Python	Ruff	Lint et formatage : plus de débat de style en revue
Python	pytest	Tests
Front	Node LTS, Vite, TypeScript	Le projet React
Front	ESLint et Prettier	Lint et formatage
Les deux	pre-commit	Lance Ruff et Prettier avant chaque commit
Les deux	GitHub Actions	À chaque PR : Ruff, pytest, build du front. Une PR rouge ne se fusionne pas

Vite plutôt que Next.js : c'est votre décision, voici l'argument. Next.js sait faire tourner du code serveur (rendu côté serveur, routes d'API), ce qui ferait double emploi avec FastAPI et ajouterait une deuxième façon de faire la même chose à apprendre. Une application React simple construite avec Vite ne fait que l'interface, et c'est React qui compte sur un CV.

LangGraph plutôt que CrewAI : un seul framework d'orchestration, pas deux. LangGraph vous fait dessiner le graphe vous-mêmes (nœuds, état, transitions) : plus de travail au début, mais vous pourrez expliquer chaque étape, là où CrewAI cache plus de choses.

Le contrat d'abord

Le front et le back ne se parlent qu'à travers l'API. Le contrat de cette API, ce sont **vos modèles Pydantic** : FastAPI en tire automatiquement un schéma OpenAPI (`/openapi.json`), et l'outil `openapi-typescript` en génère les types TypeScript du front.

Vous écrivez donc le contrat en Python, la langue que vous maîtrisez déjà. Si un champ change côté back, le front ne compile plus tant qu'il n'est pas à jour : l'erreur apparaît tout de suite, pas en production. Règle : une PR qui touche `backend/app/schemas/` régénère `frontend/src/api/schema.d.ts` dans la même PR.

Les événements du streaming font aussi partie du contrat (voir la vigilance SSE).

Développer sans la carte graphique d'Ethan

Le modèle principal tourne chez Ethan. Pour que son collègue ne dépende pas de ce PC allumé :

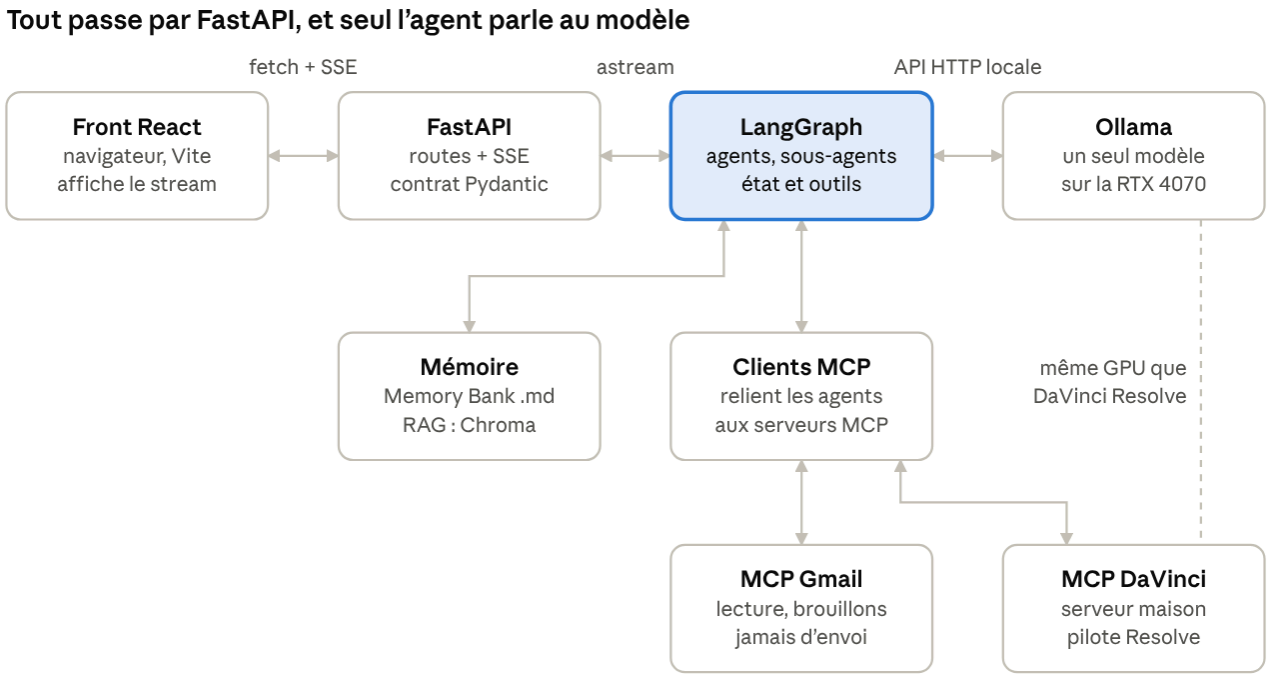
- **en développement**, chacun installe Ollama avec un petit modèle (1 à 4 milliards de paramètres). Il répond moins bien, mais l'API est la même, et le nom du modèle se règle dans `.env` ;
- **les tests n'appellent jamais de vrai modèle** : LangChain fournit de faux modèles qui renvoient des réponses écrites d'avance (`GenericFakeChatModel`). pytest tourne donc partout, CI comprise, sans GPU.

Qui fait quoi (proposition)

	Ethan	Son collègue
Domaine	« Le cerveau » : tout ce qui touche au modèle	« Le corps » : ce qui transporte et affiche
Phases pilotées	Agent LangGraph, mémoire et RAG, sous-agents, serveur MCP DaVinci	Mise en place et CI, streaming SSE, front React, MCP Gmail
Pourquoi	Le GPU est chez lui, et il connaît DaVinci	Il avance sans dépendre du GPU

- La **phase 2 (premier chat)** se fait à deux sur le même écran : c'est elle qui fixe les conventions que tout le reste suivra.
- Chaque phase a **un pilote et un relecteur**. Échangez les rôles sur au moins une phase, pour que chacun ait touché aux deux côtés.
- **Un point de 15 minutes par semaine** : ce qui est fini, ce qui bloque, les issues de la semaine.

L'arborescence du dépôt



architecture · front, API, agent, modèle, mémoire et serveurs MCP

Le navigateur ne connaît que FastAPI. FastAPI lance le graphe LangGraph, et c'est le graphe seul qui appelle Ollama, la mémoire et les outils MCP. Le trait pointillé rappelle que le modèle et DaVinci Resolve se partagent la même carte graphique.

```
conseiller/
├─ README.md                  ← la vitrine du CV : schéma, capture, comment
lancer
├─ .env.example              ← variables à copier en .env (jamais versionné)
├─ .github/workflows/ci.yml  ← Ruff + pytest + build du front à chaque PR
├─ docs/
│  ├─ decisions/             ← un .md court par choix important
│  └─ evenements-sse.md      ← la liste des événements du streaming
├─ backend/
│  ├─ pyproject.toml         ← dépendances (uv)
│  ├─ app/
│  │  ├─ main.py             ← crée l'application FastAPI
│  │  ├─ config.py           ← lit le .env (pydantic-settings)
│  │  ├─ api/                ← les routes HTTP : chat.py, memoire.py…
│  │  ├─ schemas/            ← les modèles Pydantic = le contrat avec le
front
│  │  ├─ llm/                ← le seul endroit qui parle à Ollama
│  │  ├─ agents/             ← graphes LangGraph, prompts, sous-agents
│  │  ├─ outils/             ← les fonctions que les agents peuvent appeler
│  │  ├─ memoire/            ← Memory Bank Markdown + RAG
```

```
|  |  └─ mcp/                  ← clients MCP (Gmail, DaVinci)
|  └─ tests/
├─ mcp-servers/
|  └─ davinci/                 ← votre serveur MCP (phase 8), projet Python
autonome
├─ frontend/
|  ├─ package.json
|  └─ src/
|     ├─ api/                  ← schema.d.ts généré + client de streaming
|     ├─ components/           ← Chat, Message, EtapeOutil…
|     └─ pages/
└─ data/                       ← hors Git : Memory Bank, base Chroma,
conversations
```

Trois choix à comprendre :

- **`llm/` isolé** : changer de modèle ou d'adresse Ollama se fait à un seul endroit, et c'est là que les tests branchent le faux modèle.
- **`mcp-servers/davinci/` à part** : un serveur MCP est un programme indépendant que n'importe quel client MCP peut utiliser. Il a donc son propre `pyproject.toml` et ne dépend pas du backend.
- **`data/` hors Git** : la Memory Bank contiendra des informations personnelles. Elle reste sur le PC, et le dépôt ne contient que du code.

Les 8 phases

Chaque phase se termine sur quelque chose qui marche et qu'on peut montrer. Les dates supposent environ 6 h par semaine chacun, sans pause : les examens et les vacances les décaleront. Recalez-les à la fin de la phase 2, quand vous connaîtrez votre vitesse réelle.

Phase	Dates (estimation)	Pilote	Fini quand
1. Mise en place	25/10 → 1/11	Collègue	la page affiche « ok » et la CI est verte
2. Premier chat local	2/11 → 15/11	À deux	le modèle local répond dans le navigateur
3. Streaming	16/11 → 29/11	Collègue	le texte s'affiche mot à mot, Stop arrête le GPU
4. Agent et outils	30/11 → 20/12	Ethan	l'agent appelle un outil au lieu d'inventer

Phase	Dates (estimation)	Pilote	Fini quand
5. Mémoire	21/12 → 17/1	Ethan	une info survit au redémarrage, le RAG cite sa source
6. Sous-agents	18/1 → 7/2	Ethan	le superviseur confie chaque demande au bon spécialiste
7. MCP Gmail	8/2 → 28/2	Collègue	résumé des mails, brouillon créé après validation
8. Serveur MCP DaVinci	1/3 → 28/3	Ethan	une phrase crée un marqueur dans Resolve

Phase 1 — Mise en place

- Créer le dépôt, protéger `main`, inviter l'autre, ouvrir le tableau Projects.
- `backend/` avec uv et FastAPI : une route `GET /health` qui renvoie `{"status": "ok"}`, et son test pytest.
- `frontend/` avec `npm create vite@latest` (modèle React + TypeScript) : une page qui appelle `/health`.
- La CI : Ruff, pytest et `npm run build` à chaque PR.
- Chacun installe Ollama, télécharge un modèle et discute avec lui dans un terminal (`ollama run`).

Fini quand : chacun clone le dépôt, lance le back et le front, et la page affiche « ok ».
Nouveau pour vous : Git à deux, Node et npm, le principe d'une API HTTP.

Phase 2 — Premier chat local

- Route `POST /chat` : reçoit un message, appelle Ollama avec la bibliothèque Python `ollama`, renvoie la réponse entière.
- Page React : une zone de texte, un bouton, la liste des messages.
- L'historique de la conversation est gardé côté back (une liste en mémoire suffit pour l'instant).

Fini quand : une question tapée dans le navigateur reçoit une réponse du modèle local, et la suivante tient compte de la précédente. **Nouveau** : React (composants, état avec `useState`), `fetch`, les rôles des messages d'un LLM (system, user, assistant).

Phase 3 — Le streaming

- `POST /chat/stream` envoie les tokens au fil de l'eau en SSE (voir la vigilance SSE).

- Le front lit le flux avec `fetch`, affiche le texte au fur et à mesure, et propose un bouton Stop.
- Écrire `docs/evenements-sse.md` : la liste des événements (`token`, `etape`, `erreur`, `fin`) et leurs champs.

Fini quand : le texte s'affiche mot à mot, et Stop arrête vraiment la génération (l'activité du GPU retombe aussitôt). **Nouveau** : `async` et `await` en Python, générateurs asynchrones, flux côté navigateur.

Phase 4 — Un agent avec des outils

- Réécrire le chat comme un graphe LangGraph : un nœud « modèle », un nœud « outils », et une boucle entre les deux tant que le modèle demande un outil (le schéma ReAct).
- Deux ou trois outils simples et sans risque : date et heure, lecture d'un fichier de `data/`, calcul.
- Conversations enregistrées avec un checkpointer SQLite : elles survivent au redémarrage.
- Le front affiche les étapes (« appelle l'outil date … »).

Fini quand : à « quel jour serons-nous dans 17 jours ? », l'agent appelle l'outil au lieu d'inventer, et l'interface le montre. **Nouveau** : l'appel d'outils (function calling), un graphe d'états, ce qu'est techniquement un agent.

Phase 5 — La mémoire

Deux mémoires pour deux besoins :

- **La Memory Bank** : quelques fichiers Markdown courts et stables (`profil.md`, `projets.md`, `decisions.md`), chargés à chaque fois dans le prompt système. L'agent peut proposer une modification, qui n'est écrite qu'après validation dans l'interface.
- **Le RAG** : vos documents (notes, cours, PDF) découpés en morceaux, transformés en vecteurs par un modèle d'embeddings servi par Ollama, rangés dans Chroma. Un outil `chercher_dans_mes_documents` renvoie les passages les plus proches de la question, avec leur fichier d'origine.

La Memory Bank est petite et toujours lue ; le RAG est grand et seulement cherché quand il le faut.

Fini quand : une information donnée lundi est retrouvée jeudi après redémarrage, et sur une dizaine de questions dont vous connaissez la réponse, l'agent cite le bon fichier. **Nouveau** : embeddings, similarité cosinus, découpage en morceaux (chunking), base vectorielle.

Phase 6 — Les sous-agents

- Un superviseur lit la demande et la confie à un spécialiste : planning, mails, montage. Chaque spécialiste utilise le même modèle, avec son propre prompt et ses propres outils.
- Un spécialiste reçoit seulement ce dont il a besoin, pas toute la conversation (voir la vigilance contexte).

Fini quand : sur une liste d'une vingtaine de demandes écrites à l'avance, le superviseur choisit le bon spécialiste, et vous savez expliquer celles qu'il rate. **Nouveau** : sous-graphes LangGraph, routage, évaluation simple.

Phase 7 — MCP Gmail

- Comprendre MCP : un protocole standard pour brancher des outils à un modèle. Un **serveur** MCP expose des outils ; votre backend est le **client** qui les donne à l'agent (bibliothèque `langchain-mcp-adapters`).
- Écrire vous-mêmes un petit serveur Gmail avec le SDK Python officiel, limité à trois outils : chercher, lire, créer un brouillon. Plutôt qu'un serveur tiers qui aurait accès à toute la boîte mail, vous savez exactement ce que le vôtre peut faire, et c'est l'entraînement idéal pour la phase 8.
- Aucun outil d'envoi. La création d'un brouillon attend une validation dans l'interface (`interrupt` de LangGraph).

Fini quand : « résume mes mails non lus » fonctionne, et un brouillon n'est créé qu'après un clic. **À savoir** : l'accès Gmail passe par un projet Google Cloud et OAuth. Une application laissée en mode « Test » voit son jeton expirer environ tous les 7 jours (Ethan connaît déjà ce fonctionnement avec l'API YouTube).

Phase 8 — Le serveur MCP DaVinci

- Un serveur MCP en Python (SDK officiel, `FastMCP`) qui pilote DaVinci Resolve par son API de script, livrée avec le logiciel. Piloter Resolve depuis un programme extérieur demande la version Studio (Ethan l'a) et le réglage « script externe : local » dans les préférences.
- Commencer en lecture seule (projet ouvert, timelines, durées, marqueurs), puis des actions réversibles (ajouter un marqueur, créer une timeline). Ni rendu ni suppression au début.
- Tester le serveur seul avec MCP Inspector avant de le brancher à l'agent.
- Ethan a déjà un serveur MCP DaVinci généraliste installé dans Claude : regardez ses outils pour vous inspirer, puis donnez au vôtre des outils de plus haut niveau, propres à son travail (par exemple, préparer la timeline d'une review de VOD).

- Mesurer la VRAM avec Resolve ouvert et le modèle chargé.

Fini quand : « ajoute un marqueur rouge à 1:23 nommé zoom » crée le marqueur dans Resolve. **Nouveau** : écrire un serveur MCP, l'API de script de Resolve.

Les créneaux dans la semaine

Côté Ethan, le projet prend du temps libre, pas du temps de cours ni de révisions. À partir du dimanche 25 octobre :

Jour	Moment	Durée
Mardi	soir	1 h
Jeudi	soir	1 h
Samedi	après-midi	2 h
Dimanche	après-midi	2 h

Soit 6 h par semaine. Les séances d'une heure servent aux petites tâches (une issue, une relecture de PR) ; les séances de deux heures aux morceaux qui demandent de la concentration (un nouveau concept, une intégration).

Ces créneaux sont dans le planning quotidien d'Ethan : une journée déjà trop chargée les déplace, et c'est lui qui décide de les suspendre en période d'examens.

À caler avec le collègue : au moins un créneau commun par semaine (le samedi ou le dimanche après-midi s'y prêtent), pour le point de 15 minutes, et pour travailler à deux sur les phases qui le demandent.

Vigilance : le streaming SSE entre FastAPI et React

Le principe. Avec SSE (Server-Sent Events), le serveur garde la réponse HTTP ouverte et envoie des messages texte au fil de l'eau. Chaque événement est une ligne `data: ...` suivie d'une ligne vide. Mettez-y du JSON avec un champ `type` : `token`, `etape`, `erreur`, `fin`. Décrivez chaque type avec un modèle Pydantic, pour que le format soit le même partout.

Côté FastAPI

```
import json
from fastapi import Request
from fastapi.responses import StreamingResponse
from ollama import AsyncClient

client = AsyncClient()

def evenement(type_: str, **donnees) -> str:
    return f"data: {json.dumps({'type': type_, **donnees})}\n\n"

@app.post("/chat/stream")
async def chat_stream(demande: Demande, request: Request):
    async def flux():
        try:
            reponse = await client.chat(model=MODELE,
messages=demande.messages, stream=True)
            async for morceau in reponse:
                if await request.is_disconnected():
                    break  # l'utilisateur a cliqué sur Stop
                yield evenement("token", texte=morceau["message"]["content"])
            yield evenement("fin")
        except Exception as e:
            yield evenement("erreur", message=str(e))
    return StreamingResponse(flux(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache"})
```

Côté React

```
const controleur = new AbortController(); // controleur.abort() = bouton Stop
const reponse = await fetch("/api/chat/stream", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ messages }),
  signal: controleur.signal,
});
const lecteur = reponse.body!.pipeThrough(new
TextDecoderStream()).getReader();
let tampon = "";
while (true) {
  const { value, done } = await lecteur.read();
  if (done) break;
  tampon += value;
  const blocs = tampon.split("\n\n");
  tampon = blocs.pop()!; // le dernier bloc peut être incomplet
```

```
  for (const bloc of blocs) {
    if (!bloc.startsWith("data: ")) continue;
    const evt = JSON.parse(bloc.slice(6));
    if (evt.type === "token") setTexte((t) => t + evt.texte);
  }
}
```

Les pièges

- **`EventSource` ne suffit pas.** L'API SSE intégrée au navigateur ne fait que des `GET` sans corps, alors qu'un chat envoie un `POST` avec les messages. D'où `fetch` et la lecture du flux ci-dessus.
- **Un événement peut arriver coupé en deux.** Le réseau découpe le flux où il veut : on accumule dans un tampon et on ne traite que les blocs terminés par une ligne vide.
- **Ne jamais bloquer la boucle asynchrone.** Un appel synchrone (`ollama.chat` classique) dans une fonction `async def` bloque toutes les autres requêtes pendant la génération. Utilisez `AsyncClient`, puis `astream` avec LangGraph.
- **Tout arrive d'un coup à la fin** : un intermédiaire met le flux en tampon. Suspects habituels : une compression GZip appliquée à la route, un proxy. L'onglet Réseau des outils de développement montre si les morceaux arrivent un par un.
- **Une erreur au milieu du flux ne change plus le code HTTP**, déjà envoyé (200) : elle doit voyager comme un événement `erreur`, que le front affiche.
- **Stop doit arrêter le GPU, pas seulement l'affichage.** `abort()` ferme la connexion ; côté serveur, la boucle s'arrête et l'appel à Ollama est abandonné. Vérifiez que l'activité du GPU retombe.
- **React et les re-rendus** : la forme `setTexte((t) => t + ...)` évite de perdre des tokens. En développement, le mode strict de React lance les effets deux fois : démarrez le flux depuis le clic sur Envoyer, pas dans un `useEffect`.
- **CORS** : en développement, le proxy de Vite (`server.proxy` dans `vite.config.ts`) fait passer `/api` vers FastAPI ; le navigateur croit parler à un seul serveur, et il n'y a rien à configurer.
- **Avec LangGraph (phase 4)** : `graph.astream(..., stream_mode=["messages", "updates"])` donne à la fois les tokens (`messages`) et la fin de chaque nœud (`updates`). Les premiers deviennent des événements `token`, les seconds des événements `etape`.

Vigilance : contexte et VRAM en local

Le contexte, c'est tout ce que le modèle lit à chaque appel : prompt système, Memory Bank, passages du RAG, historique, résultats d'outils, et la réponse qu'il est en train d'écrire. Sa taille maximale se règle avec `num_ctx`.

Piège n°1 : la troncature silencieuse

La valeur par défaut de `num_ctx` dans Ollama est courte (quelques milliers de tokens selon la version). Quand le contexte déborde, Ollama coupe le début **sans erreur** : le modèle « oublie » son prompt système ou la Memory Bank, et vous cherchez le bug dans votre code. Fixez toujours `num_ctx` vous-mêmes (`options={"num_ctx": 16384}`), et enregistrez à chaque appel les compteurs que renvoie Ollama : `prompt_eval_count` (tokens lus) et `eval_count` (tokens écrits). C'est la seule façon de savoir à quel point le contexte est plein.

Combien coûte le contexte en VRAM

Pour chaque token du contexte, le modèle garde en VRAM ses clés et ses valeurs d'attention : c'est le **cache KV**.

`octets par token = 2 (clé et valeur) × couches × têtes KV × dimension par tête × octets par nombre`

Exemple avec Llama 3.1 8B (32 couches, 8 têtes KV, dimension 128, en 16 bits) : 2 × 32 × 8 × 128 × 2 = 128 Kio par token, donc **1 Gio pour 8k tokens et 4 Gio pour 32k**. Les chiffres de chaque modèle sont dans son `config.json` sur Hugging Face, et varient du simple au double d'un modèle à l'autre.

Les réglages Ollama à connaître

Réglage	Effet	Conseil
`OLLAMA_FLASH_ATTENTION=1` et `OLLAMA_KV_CACHE_TYPE=q8_0`	Cache KV stocké sur 8 bits : moitié moins de VRAM, perte de qualité faible	À activer dès que le contexte grandit
`OLLAMA_NUM_PARALLEL`	Requêtes traitées en même temps ; chacune réserve son propre contexte	1 : vos agents parlent l'un après l'autre
`OLLAMA_MAX_LOADED_MODELS`	Nombre de modèles gardés en VRAM	Petit, pour ne jamais en charger deux gros

Réglage	Effet	Conseil
`keep_alive`	Durée de présence en VRAM après le dernier appel (5 min par défaut)	`0` pour libérer la carte tout de suite, ou `ollama stop`

Pour vérifier : `ollama ps` doit afficher « 100% GPU ». Un partage CPU/GPU veut dire que le modèle déborde sur la RAM : réduisez `num_ctx` ou prenez un modèle plus petit.

Un budget de contexte par appel (proposition pour 16k)

Partie	Budget	Si ça dépasse
Prompt système et Memory Bank	2k	Raccourcir la Memory Bank : elle doit rester un résumé
Passages du RAG	4k	Moins de passages, ou des morceaux plus courts
Historique	6k	Garder les derniers messages et résumer le reste (`trim_messages` , ou un nœud de résumé)
Résultats d'outils	2k	Tronquer ou résumer avant de les donner à l'agent : un seul mail peut dépasser
Réponse	2k	Demander une réponse plus courte

Deux habitudes de plus :

- **Les sous-agents ne reçoivent pas toute la conversation.** Le superviseur leur passe une consigne courte, et ils rendent un résultat court.
- **Le début du prompt ne bouge pas.** Ollama réutilise le début du prompt quand il est identique à l'appel précédent : gardez le prompt système et la Memory Bank en tête, dans le même ordre, et les réponses démarrent plus vite.

DaVinci Resolve et le modèle sur la même carte

Pas de donnée pour cette configuration avant de mesurer. En phase 8, relevez la VRAM avec `nvidia-smi` dans quatre situations : rien d'ouvert ; Resolve avec un projet de montage habituel ; Resolve et le modèle chargé ; pendant un rendu. Si ça ne tient pas, trois parades : décharger le modèle avant un rendu (`keep_alive` à 0), passer à un modèle plus petit pendant le montage, ou réduire `num_ctx` .

Apprendre en construisant

La méthode : pour chaque notion nouvelle, suivez le tutoriel officiel jusqu'au point dont la phase a besoin, pas plus loin, puis construisez. Notez en quelques lignes ce que vous avez compris dans `docs/` : ce sont vos réponses d'entretien.

Claude peut expliquer une notion, débloquer une erreur ou relire une PR. Écrivez quand même le code vous-mêmes : c'est lui que vous devrez expliquer.

Phase	Ressource	Pour comprendre
1	Pro Git, en français	Branches, fusion, conflits (chapitres 1 à 3)
1	GitHub flow	Le cycle branche, PR, revue, fusion
1	Documentation uv	Projet Python, dépendances, verrouillage
1	Tutoriel FastAPI	Routes, modèles Pydantic, documentation automatique
1	Guide Vite	Créer et lancer le projet front
1	GitHub Actions	La CI qui teste chaque PR
2	Guide JavaScript du MDN	Le langage, en partant de Python
2	Manuel TypeScript	Les types côté front
2	react.dev/learn	Composants, props, état
2	Bibliothèque Python d'Ollama et sa documentation	Appeler le modèle, ses options
2	Cours LLM de Hugging Face	Ce qui se passe dans un modèle : tokens, attention
3	Server-sent events sur le MDN	Le format SSE
3	API Streams sur le MDN	Lire un flux dans le navigateur
3	Concurrence et async dans FastAPI	Pourquoi ne jamais bloquer la boucle

Phase	Ressource	Pour comprendre
4	Introduction to LangGraph, LangChain Academy	Le cours gratuit de référence : graphes, outils, mémoire, streaming
4	Documentation LangGraph	La référence pendant que vous codez
4	Appel d'outils avec Ollama	Comment un modèle demande un outil
5	Modèles d'embeddings avec Ollama	Transformer un texte en vecteur
5	Documentation Chroma	Ranger et chercher des vecteurs
5	Cookbook Hugging Face	Des exemples de RAG complets
6	Systèmes multi-agents dans LangGraph	Superviseur, sous-graphes, passage de relais
7	modelcontextprotocol.io	Ce qu'est MCP, puis le guide « construire un serveur » en Python
7	SDK Python MCP	Écrire un serveur avec `FastMCP`
7	langchain-mcp-adapters	Donner les outils MCP à un agent LangGraph
7	Démarrage rapide de l'API Gmail en Python	OAuth et premiers appels
7	Human-in-the-loop dans LangGraph	Mettre l'agent en pause pour une validation
8	MCP Inspector	Tester un serveur MCP sans agent
8	Le README de script livré avec Resolve : `C:\ProgramData\Blackmagic Design\DaVinci Resolve\Support\Developer\Scripting\README.txt`	Toutes les fonctions de l'API de Resolve

Questions encore ouvertes

À trancher à deux, idéalement avant la fin de la phase 1 :

- [ ] **Le nom définitif** du projet, avant de créer le dépôt.
- [ ] **Vite ou Next.js** pour le front : l'argument est plus haut, la décision vous appartient.
- [ ] **Le rythme du collègue** : combien d'heures par semaine, quels créneaux en commun, et a-t-il une carte graphique ?
- [ ] **Les données de chacun** : la Memory Bank et les documents du RAG sont personnels. Chacun fait-il tourner sa propre instance avec son propre dossier `data/` ?
- [ ] **Le compte Gmail** que l'agent lira en phase 7.
- [ ] **Public ou privé à la fin** : pour un CV, un dépôt public avec une licence (MIT, par exemple), un README soigné et une courte vidéo de démonstration pèsent plus qu'un lien privé.